



TECHNISCHE ÜBERGABE · DOKUMENT 01

Übergabe-Dokument für das BVDW-Team

Technische Übergabe des Foundation Models
Atlas an den BVDW — inkl. Account-Setup,
Migrations-Anleitung und
Betriebsempfehlungen.

Projekt: Foundation Models Atlas · AI Tech Lab

Stand: 06. Oktober 2026

Aktuelle URL: bosses-foundation-models.vercel.app

Empfänger: BVDW IT / AI Tech Lab Koordination **Übergeber:** Bosse Küllenberg (Lead AI Tech Lab)
Stand: April 2026 **Projekt-URL (aktuell):** <https://bosses-foundation-models.vercel.app>

Executive Summary

Der Foundation Models Atlas ist eine neutrale, kuratierte Übersicht relevanter AI-Foundation-Models mit europäischer Souveränitätsperspektive. Er besteht aus einem schlanken Frontend (statisches HTML/CSS/JS), fünf Serverless-API-Routen und einer Supabase-Datenbank mit 264 Modellen (Stand Update Oktober 2026), 120 Vendors, 80 Lizenzen und einer vollständigen Versionshistorie.

Kernfunktionen:

- Katalog mit 103 konversationellen LLMs und 45 spezialisierten Modellen (Bild, Audio, Video)
- Drei Ansichten: **Liste**, **Quartett-Karten** (mit PDF-Export) und **Swipe-Finder**
- Transparente Bewertung pro Modell mit Info-Tooltips und einer kompletten Methodik-Legende
- **Deployment-Buckets** (B1-B5) von „Eigene Hardware“ bis „Vendor-Direct US-Cloud“
- Passwortgeschützter **Edit-Mode** mit Versionshistorie, Rollback und Neuanlage-Funktion
- **Deeplinks** pro Modell, Teilen via Web-Share-API oder Zwischenablage
- Link auf Hugging-Face-Model-Card für aktuelle Benchmark-Werte (keine eigene Pflege)

Das Projekt läuft heute auf einem privaten Vercel- und Supabase-Account und soll an das BVDW übergeben werden, damit es unter der Verbands-Hoheit weiterbetrieben werden kann.

1. Empfehlung: Hybrid-Betrieb mit SaaS-Komponenten

TL;DR: Das BVDW übernimmt den Code als Git-Repo, legt eigene Accounts bei Vercel und Supabase an, und betreibt den Atlas mit ~0 € monatlichen Kosten im Free-Tier beider Dienste.

Warum Hybrid-Betrieb die richtige Wahl ist

Die Daten dieses Atlas sind **öffentlich und nicht sensibel**. Es gibt keine personenbezogenen Daten außer den freiwillig eingetragenen Editor-Namen in der Versionshistorie. Damit entfällt der DSGVO-Zwang, die Datenbank selbst zu hosten — und damit der Großteil des Admin-Aufwands, der typischerweise gegen SaaS spricht.

Die relevanten Services:

KOMPONENTE	EMPFEHLUNG	BEGRÜNDUNG
Code-Hosting (Git)	GitHub (BVDW-Org) oder GitLab	Standard, Team-Kollaboration, Integration mit Vercel
Hosting Frontend + API	Vercel (Free-Tier)	Kostenlos für statische Sites + 100 GB Bandbreite/Monat + unbegrenzte Serverless Functions
Datenbank (Postgres)	Supabase (Free-Tier)	500 MB DB, 2 GB Bandbreite — reicht für die 264 Modelle um Faktor 50
Domain	BVDW-eigen (z.B. <code>atlas.bvdw.org</code>)	Optional, über Vercel-DNS einzurichten
Externe CDNs für Logos/Flaggen	bleiben extern (lobehub, flagcdn, simple-icons)	Keine eigene Pflege nötig, alle öffentlich + frei lizenziert

Kostenschätzung für das BVDW:

- Free-Tier-Betrieb: **0 €/Monat**
- Vercel Pro (falls Passwortschutz nötig): 20 €/Monat/Mitglied
- Supabase Pro (falls > 500 MB DB oder Point-in-Time-Recovery): 25 €/Monat
- Domain: ca. 15 €/Jahr

Realistisch startet das BVDW komplett im Free-Tier und entscheidet später, ob ein Paid-Plan nötig wird.

Wichtiger Betriebshinweis — Supabase-Pause im Free-Tier: Supabase pausiert Free-Tier-Projekte nach **7 Tagen ohne Datenbankaktivität** automatisch. Im pausierten Zustand ist die Projekt-Subdomain offline (DNS NXDOMAIN), die API liefert HTTP 500, das statische Frontend läuft aber weiter. Die Daten gehen **nicht** verloren — sie sind nur heruntergefahren. Reaktivierung erfolgt im Supabase-Dashboard („Restore project“, 2–5 Minuten) oder per Management-API (`POST https://api.supabase.com/v1/projects/<ref>/restore` mit einem Personal-Access-Token, dauert ca. 3 Minuten bis `ACTIVE_HEALTHY`).

Vorfall Oktober 2026: Genau dieser Fall ist eingetreten — das Projekt war pausiert, die Live-Seite zeigte keine Modelle. Ursache: Der Vercel-Cron wurde im Mai 2026 eingerichtet, als das Projekt bereits pausiert war, und konnte es nicht reaktivieren; ein 503 des Keep-Alive fiel niemandem auf. Konsequenz: zusätzlich zum Cron gibt es jetzt einen **GitHub-Actions-Workflow** (`.github/workflows/keep-alive.yml` , zweimal täglich), der bei HTTP ≠ 200 fehlschlägt und damit eine E-Mail an den Repo-Owner auslöst. Nach jeder Reaktivierung einmal `curl https://<domain>/api/keep-alive` prüfen.

Drei Gegenmaßnahmen, je nach Nutzungsprofil:

1. Keep-Alive-Cron (kostenlos, bereits eingebaut): Eine Vercel-Cron-Funktion (`/api/keep-alive`) pingt die Datenbank täglich um 06:00 UTC. Solange das Vercel-Deployment aktiv ist, wird das Projekt nie 7 Tage inaktiv und pausiert nicht. Reicht für den normalen Betrieb vollständig aus.

2. GitHub-Actions-Keep-Alive mit Alarm (kostenlos, eingebaut seit Oktober 2026):

Unabhängig von Vercel ruft ein Workflow den Keep-Alive-Endpunkt zweimal täglich auf und schlägt fehl, wenn die Datenbank nicht antwortet — GitHub benachrichtigt dann per E-Mail. Damit ist der Ausfall vom Mai/Oktobre 2026 (stiller 503) nicht mehr möglich.

3. Supabase Pro (25 €/Monat): Pausiert grundsätzlich nie, zusätzlich tägliche Backups und Point-in-Time-Recovery. Empfehlenswert, sobald der Atlas offiziell unter bvdw.org läuft und ein Ausfall sichtbar wäre.

Für die produktive BVDW-Phase empfehle ich: **Keep-Alive aktiv lassen UND mittelfristig auf Supabase Pro gehen** — der Cron ist die Versicherung gegen Pause, Pro ist die Versicherung gegen alles andere (Backup, Recovery, SLA).

Was das BVDW NICHT selbst hosten sollte

- **Supabase selbst:** Grundsätzlich möglich (Docker-Compose-Stack), aber für wenige hundert Datensätze extremer Overkill. Der Betrieb eines produktiven Postgres mit Backups, Upgrades, Monitoring kostet mehr DevOps-Zeit als er rechtfertigt.
- **Vercel-Alternative:** Der Atlas ist stark auf Vercels Routing und Serverless-Functions optimiert. Ein Umzug auf z.B. eigenen Node.js-Server hinter nginx wäre möglich, aber nicht kostenfrei.
- **CDNs für Logos und Flaggen:** `cdn.jsdelivr.net` , `flagcdn.com` , `api.dicebear.com` und `logo.clearbit.com` sind frei und performant. Eigene Bereitstellung dieser Assets wäre nur sinnvoll, wenn eine BVDW-spezifische Offline-Distribution (z.B. Print) gewünscht wird.

Was das BVDW selbst kontrolliert

Auch im SaaS-Modus hat das BVDW volle Hoheit über:

- **den Code** (eigenes Git-Repo, eigene Commits, eigene Contributor-Rechte)
 - **die Daten** (eigene Supabase-Instanz, eigene Export-Rechte, eigenes Datenmodell)
 - **die Domain** (eigenes `atlas.bvdw.org` oder gleichwertig)
 - **die Passwörter und Zugriffe** (eigenes Edit-Password, eigene Vercel-Team-Mitglieder)
 - **die Inhalte** (Editorial via Edit-Mode, vollständige Versionshistorie)
 - **die Ausgangsdaten** (es gibt ein Export-Script, das die vollständige Datenbank als JSON extrahiert — jederzeit ein Point-in-Time-Backup erzeugbar)
-

2. Tech-Stack

Stack-Entscheidungen, damit das BVDW-Team weiß, womit sie es zu tun haben:

Frontend (statisch)	
└─ index.html	Single-Page-Shell
└─ styles.css	ca. 950 Zeilen
└─ app.js	ca. 1800 Zeilen Vanilla JS (kein Build-Step)
Backend (Vercel Serverless Functions, Node.js 20)	
└─ api/models.js	GET – alle Modelle eines Atlas
└─ api/meta.js	GET – Meta + Filter-Werte + Lizenzen
└─ api/history.js	GET – Versions-Feed oder Einzel-Historie
└─ api/auth.js	POST – Passwort-Check
└─ api/edit.js	POST – Modell aktualisieren + Snapshot
└─ api/create.js	POST – Neues Modell anlegen
└─ api/delete.js	POST – Soft-Delete
└─ api/revert.js	POST – Version wiederherstellen
Datenbank (Supabase Postgres)	
└─ vendors	Hersteller-Stammdaten + Logo-URLs
└─ models	Modell-Daten (JSONB flexibel)
└─ model_versions	Versionshistorie pro Edit
└─ licenses	Lizenz-Katalog
└─ atlas_meta	Filter-Definitionen pro Atlas
Assets	
└─ bvdw-logo/	BVDW-Corporate-PNGs (4 Varianten)
└─ Externe CDNs	lobehub/icons, flagcdn, simple-icons, dicebear

Kein Build-Step, keine Compilation, keine Node-Module im Produktiv-Build. Das Frontend ist direkt lesbar und editierbar, die API-Functions nutzen nur zwei NPM-Pakete (`@supabase/supabase-js` und `pg` für Migrations).

3. Was das BVDW vorbereiten muss

Konten, die das BVDW anlegen sollte

1. GitHub-Organisation (oder GitLab-Gruppe) — z.B. `bvdw-ai-tech-lab`

- Public oder Private — beides funktioniert. Public würde transparenter wirken und externes Feedback erlauben.
- Ein technischer Account-Admin mit Admin-Rechten
- Mind. 2 Maintainer-Rechte für das AI Tech Lab

2. Vercel-Team-Account auf `vercel.com`

- Free-Tier reicht für den Start
- Upgrade auf Pro (20 €/Monat) nur wenn Password-Protection oder Analytics gewünscht
- GitHub-Integration aktivieren, damit automatische Deployments bei Pushes

3. Supabase-Organization auf supabase.com

- Free-Tier reicht
- Region: `eu-central-1` (Frankfurt) — für Latenz und Datenhoheit
- Upgrade auf Pro (25 €/Monat) später optional für tägliche Backups und Point-in-Time-Recovery

4. Domain-Eintrag (optional aber empfohlen)

- Subdomain von bvdw.org, z.B. `atlas.bvdw.org` oder `foundationmodels.bvdw.org`
- DNS: CNAME auf Vercel (Details im Vercel-Dashboard nach Projekt-Anlage)

Personelle Zuständigkeiten

- **1 Tech-Owner** beim BVDW für Deploys und Wartung (1-2 Stunden pro Monat)
- **3-8 redaktionelle Kolleg:innen** mit Edit-Password für inhaltliche Pflege

4. Setup-Anleitung Schritt für Schritt

Schritt 1 — Code-Repository anlegen

Der aktuelle Projektordner liegt unter `/Users/macbosse/Library/Mobile Documents/com~apple~CloudDocs/PAIDAI/BVDW` und kann als Ausgangsbasis dienen. Folgende Dateien sind im Übergabe-Paket enthalten:

<code>index.html, styles.css, app.js</code>	Frontend
<code>api/**</code>	Serverless Functions
<code>scripts/**</code>	Migrations- und Wartungs-Scripts
<code>supabase/schema.sql</code>	Datenbank-Schema
<code>package.json</code>	Dependencies
<code>vercel.json</code>	Deployment-Konfiguration
<code>HANDOVER_BVDW.md</code>	dieses Dokument
<code>HANDOVER.md</code>	Team-Dokumentation (Edit-Workflow)
<code>assets/bvdw-logo/</code>	BVDW-Logo-PNGs
<code>data-export/</code>	vollständiger Datenbank-Export (optional)
<code>models.json</code>	Quell-Daten (als Referenz)
<code>specialized_models.json</code>	dto.
<code>.gitignore</code>	korrekt gesetzt, ignoriert <code>.env.local</code> und
<code>Secrets</code>	

NICHT im Repo sollten sein:

- `.env.local` (enthält persönliche Supabase-Zugänge)
- `node_modules/`
- `.vercel/`

Diese sind alle bereits in `.gitignore` eingetragen.

Schritte:

1. Lokal: `git init && git add . && git commit -m "Initial handover of BVDW Foundation Models Atlas"`
2. Neues GitHub-Repo anlegen (z.B. `bvdw-ai-tech-lab/foundation-models-atlas`, privat oder public)
3. `git remote add origin git@github.com:bvdw-ai-tech-lab/foundation-models-atlas.git`
4. `git branch -M main && git push -u origin main`

Schritt 2 — Supabase-Instanz einrichten

1. Im Supabase-Dashboard: „**New Project**“, Region `eu-central-1`, Password sicher generieren und speichern
2. Nach ~2 Minuten Setup-Zeit: Settings → API → **Project URL**, **anon key** und **service_role key** notieren (die werden gleich gebraucht)
3. Settings → Database → **Connection String** für Direktzugriff kopieren
4. SQL-Editor öffnen, den Inhalt von `supabase/schema.sql` einfügen und ausführen. Das legt die 5 Tabellen mit RLS-Policies an.

Schritt 3 — Daten migrieren

Die aktuelle Datenbank enthält:

- 96 Vendors (inkl. Logo-URLs und Metadaten)
- 264 Modelle (mit vollständigen Feldern: Vendor, Region, Tier, Souveränität, Einsatz-Optionen, Sprachen, Modalitäten, Insider-Notizen ...)
- ~330 Versions-Snapshots (die komplette Historie aller Edits seit dem Start)
- 56 Lizenzen
- 2 Atlas-Metadaten-Einträge

Transfer-Pfad (empfohlen):

1. Im aktuellen Projekt: `npm run export-all` erzeugt den Ordner `data-export/` mit 5 JSON-Files
2. In die neue Supabase-Instanz ist das Schema schon via Schritt 2 angelegt
3. In der neuen Umgebung: `.env.local` mit den BVDW-Supabase-Credentials ausstatten und `npm run import-all` ausführen
4. Das importiert alle JSONs via Service-Role-Key in die neue Instanz, idempotent

Alternative (klassisch, ohne meine Scripts):

- `pg_dump` aus der alten Supabase, `pg_restore` in die neue. Weniger portabel, aber Postgres-nativer. Für den BVDW-Use-Case aber nicht nötig — die Export/Import-Scripts decken alles ab.

Schritt 4 — Vercel-Projekt anlegen

1. Auf `vercel.com`: „**Import Git Repository**“ → das neue GitHub-Repo auswählen
2. Framework-Preset: „**Other**“ (Vercel erkennt das static + serverless-Pattern automatisch)
3. Build Command und Output Directory leer lassen (wird nicht gebraucht)
4. Vor dem ersten Deploy: Environment Variables setzen (Settings → Environment Variables):

```
SUPABASE_URL           = https://xxx.supabase.co           (aus Supabase)
SUPABASE_SERVICE_ROLE_KEY = eyJhbGciOi...                 (aus Supabase, geheim!)
EDIT_PASSWORD          = neues-bvdw-passwort              (frei wählbar)
```

Alle drei als „**Production**“ und optional auch als „Preview“ und „Development“.

5. **Deploy** klicken. Nach ~30 Sekunden ist die Seite unter `<projektname>.vercel.app` erreichbar.

Schritt 5 — Custom Domain einrichten (optional)

1. Vercel-Dashboard → Settings → Domains → „**Add Domain**“
2. Gewünschte Subdomain eintragen (z.B. `atlas.bvdw.org`)
3. Vercel zeigt einen CNAME-Eintrag, der im `bvdw.org`-DNS zu setzen ist
4. Propagation dauert typischerweise 5 Minuten bis 24 Stunden

Schritt 6 — Smoke-Test

Nach dem Deploy:

- ☐ Homepage lädt und zeigt Liste der Modelle
 - ☐ Filter in der Sidebar funktionieren (Region, Tier, Buckets)
 - ☐ Quartett-Modus zeigt Karten mit Logos
 - ☐ Finder-Modus ist durchlaufbar
 - ☐ Detail-Modal zeigt Modell-Infos mit allen Info-Tooltips
 - ☐ Edit-Mode-Aktivierung mit dem neuen Passwort funktioniert
 - ☐ Ein Test-Edit und anschließender Revert laufen durch
 - ☐ PDF-Download einer Quartett-Karte funktioniert
-

5. Wartung und laufender Betrieb

Für die redaktionelle Pflege (BVDW-Team)

Das gesamte tägliche Geschäft läuft über den **Edit-Mode**:

1. `atlas.bvdw.org` öffnen, oben rechts „Edit Mode“ klicken
2. Passwort und eigenen Namen eintragen
3. Auf einer Karte den Stift-Button anklicken → alle Felder editieren
4. „Neues Modell“ in der oberen Leiste, um Modelle zu ergänzen
5. „Historie“ in der Kopfzeile zeigt alle Änderungen mit Revert-Option

Die komplette Edit-Workflow-Dokumentation liegt in `HANDOVER.md` (ebenfalls im Repo).

Für den technischen Owner

Passwort rotieren:

```
vercel env rm EDIT_PASSWORD production
printf "%s" "neues-passwort" | vercel env add EDIT_PASSWORD production
vercel --prod --yes
```

Backup erzeugen:

```
npm run export-all
# Schreibt nach data-export/ – in Git committen oder extern archivieren
```

Größere Daten-Updates (viele neue Modelle, Legacy-Markierungen):

```
npm run import-delta -- data/updates/2026-10-update.json --dry-run # Plan anzeigen
npm run import-delta -- data/updates/2026-10-update.json           # einspielen
npm run export-all                                               # Backup
aktualisieren
```

Ein Update-Paket in `data/updates/` enthält neue Modelle, Lizenzen, Vendors, Patches (z.B. `status: legacy` + `successor`) und Soft-Deletes. Das Script ist idempotent und schreibt für jede Änderung einen Versions-Snapshot — exakt so, als hätte ein Redakteur im Edit-Mode gearbeitet. Das Paket vom Oktober 2026 ist das Referenzbeispiel. Ältere Schema-Migrationen: `scripts/migrate-phase*.js`.

Logo-Resolver neu starten (wenn Lobe-Icons aktualisiert wurden):

```
npm run resolve-logos
```

Deploy-Workflow

Für Code-Änderungen:

1. Lokal: `npx vercel dev` (lädt Supabase-Credentials aus `.env.local`) oder ohne Secrets `node scripts/dev-static.js` (statisches Frontend, API-Aufrufe gehen an die Produktion)
2. Git-Push auf `main` → Vercel deployt automatisch
3. Preview-Deploys auf Feature-Banches werden ebenfalls automatisch erstellt

Für Daten-Änderungen: Kein Deploy nötig. Die Daten liegen in Supabase und werden live gelesen. Änderungen im Edit-Mode sind sofort live.

6. Datenschutz und Compliance

Welche Daten gespeichert werden

- **Modell-Metadaten:** alle öffentlich bekannt (Name, Vendor, Pricing, etc.)
- **Editor-Namen:** freiwillig eingetragen beim Aktivieren des Edit-Modus. Können Klartextnamen oder Pseudonyme sein.
- **Edit-Summaries:** freiwillige Kommentare zu Änderungen
- **Timestamps:** automatisch bei jeder Änderung

Es werden **keine IP-Adressen**, Session-Tracker oder Cookies gespeichert. Vercel-Analytics sind standardmäßig deaktiviert — sollten sie aktiviert werden, wäre ein Hinweis in der Datenschutzerklärung des BVDW nötig.

DSGVO-Relevanz

Die einzigen potenziell personenbezogenen Daten sind die Editor-Namen. Empfehlung:

- Entweder explizit darauf hinweisen, dass das Team Pseudonyme nutzen darf
- Oder in der Datenschutzerklärung die Aufbewahrung dieser Daten als Teil der redaktionellen Arbeit begründen (Art. 6 Abs. 1 f DSGVO — berechtigtes Interesse)

Auftragsverarbeitungsverträge

- **Vercel:** Data Processing Addendum ist über das Pro-Plan verfügbar. Für Free-Tier gelten die Standardnutzungsbedingungen.
- **Supabase:** bietet DPA auf Pro-Plan, EU-Hosting-Region `eu-central-1` ist verfügbar.

Für die Daten im aktuellen Scope ist weder ein DPA zwingend noch ein EU-Hosting rechtlich erforderlich, weil keine personenbezogenen Daten in nennenswertem Umfang verarbeitet werden. Wenn der BVDW konservativ entscheiden will, lohnt sich trotzdem der Sprung auf Pro + DPA für beide Services.

7. Alternativen zu Vercel und Supabase (falls gewünscht)

Falls das BVDW bestimmte Komponenten selbst hosten möchte:

Alternative zu Vercel

OPTION	AUFWAND	EIGNUNG
Netlify	niedrig	Funktioniert identisch, API-Functions leicht unterschiedlich
Cloudflare Pages + Workers	mittel	Native EU-CDN, aber Serverless-Functions müssen umgeschrieben werden
Eigener nginx + Node.js	hoch	Frontend als Static-Serve, API-Funktionen als Express-App umschreiben

Alternative zu Supabase

OPTION	AUFWAND	EIGNUNG
Self-Hosted Supabase (Docker)	hoch	Funktioniert, erfordert dauerhaften DevOps-Verantwortlichen
Eigener Postgres + PostgREST	hoch	Volle Kontrolle, aber keine Row-Level-Security-UX, keine Auth-Integration
Neon, Xata, PlanetScale	niedrig	SaaS-Alternativen, aber keine direkten RLS-Policies wie Supabase

Meine klare Empfehlung: Bei Supabase-SaaS bleiben, weil die Data-Sensibilität nicht rechtfertigt, diese Admin-Last auf das BVDW zu legen. Wenn sich das ändert (z.B. weil zukünftig sensible Daten drin landen), ist ein Umzug planbar.

8. Offene Punkte und Roadmap

Bereits gebaut, aber erweiterbar

- **AI-generierte Hero-Bilder** pro Modell — ursprünglich geplant, aktuell via CSS-Gradient als Fallback. Kann später über ein FLUX- oder Stable-Diffusion-Script ergänzt werden.
- **Bulk-Export als ZIP** aller PDF-Karten — aktuell nur einzelne Karten als PDF, ZIP-Bulk wäre ~30 Zeilen Code

- **Audit-Script für tote Logo-URLs** — ein wöchentlicher Healthcheck wäre sinnvoll. Der Clearbit-Logo-Dienst (`logo.clearbit.com`) ist 2026 abgeschaltet; betroffene `image_url` -Werte wurden im Oktober 2026 auf das Vendor-Logo zurückgesetzt.
- **Entscheidungsmodelle (System One / Decision AI)** — seit Oktober 2026 als Task-Typ `decision` und Modellklasse `system-one` im Spezial-Atlas geführt; wächst die Klasse auf 10+ relevante Modelle, ist ein eigener Atlas-Tab mit eigenem Schema (Kalibrierung, Latenz, Schema-Support) die saubere nächste Stufe — siehe `docs/system-one-decision-models.md` .

Potenzielle Erweiterungen

- **Team-Kommentare pro Modell** (ohne Überschreiben der Hauptdaten)
- **Benachrichtigungen** bei Edits (E-Mail oder Slack-Webhook)
- **Public-API** falls andere Verbände oder Firmen auf den Atlas zugreifen wollen
- **Multi-Language** (EN-Version des Interfaces)

Alle diese Erweiterungen sind bewusst offen gelassen — BVDW entscheidet nach Use-Case-Bedarf.

9. Support und Kontakt

Bei technischen Rückfragen während der Übergabe

Ansprechpartner: **Bosse Küllenberg**, b.kuellenberg@gmail.com Verfügbarkeit: Q2 2026 für 1-2 Handover-Sessions à 60 Minuten

Dokumentation im Repo

- `HANDOVER_BVDW.md` — dieses Dokument (technische Übergabe)
- `HANDOVER.md` — für das redaktionelle Team, wie der Edit-Mode funktioniert
- `supabase/schema.sql` — kommentiertes Datenbank-Schema
- `scripts/` — alle Migrations- und Wartungs-Scripts, mit Inline-Kommentaren

Externe Abhängigkeiten

SERVICE	ROLLE	SLA-RISIKO
Vercel	Hosting	enterprise-grade, marginal
Supabase	DB	enterprise-grade auf Pro-Plan, best-effort auf Free
cdn.jsdelivr.net	JS-Libraries	hoch zuverlässig
flagcdn.com	Flaggen-SVGs	frei, Best-Effort
unpkg.com/@lobehub/icons	Vendor-Logos	frei, Best-Effort
api.dicebear.com	Logo-Fallback	frei, Best-Effort
cdn.simpleicons.org	Brand-Icons	frei, Best-Effort

Keiner der externen Dienste enthält personenbezogene Daten. Bei Ausfall externer CDNs bleibt der Atlas funktional (Fallbacks sind im Code implementiert), nur optisch eingeschränkt.

10. Checkliste für den Übergabe-Abschluss

Das BVDW-Team kann gegen diese Liste abhaken:

- ☐ GitHub-Repo angelegt, Code übernommen
- ☐ Supabase-Projekt angelegt, Schema ausgeführt
- ☐ Daten importiert (via `npm run import-all`)
- ☐ Vercel-Projekt angelegt, ENV-Variablen gesetzt
- ☐ Erster Deploy erfolgreich, Seite erreichbar
- ☐ Custom-Domain konfiguriert (optional)
- ☐ Edit-Mode-Passwort im Team geteilt (via Passwort-Manager)
- ☐ Smoke-Test durchlaufen
- ☐ Backup-Routine eingerichtet (wöchentlicher `export-all`)
- ☐ Alter Betrieb (bosses-foundation-models.vercel.app) kann abgeschaltet werden

Nach Abschluss dieser Liste ist das BVDW vollständig autark im Betrieb des Atlas.

Ende des Übergabe-Dokuments. Bei Fragen jederzeit gerne Rückmeldung.